

Phantom Loader — Malware Analysis Write-up

1. Introduction

Phantom Loader is a 64-bit Windows malware sample with multiple anti-analysis checks, an obfuscated C2 configuration, PE loading functionality, and client fingerprinting.

The goal of this challenge was to use **Ghidra** to find 15 specific values from the malware. Some values were directly visible, while others had to be found by following functions, checking API arguments, or decoding hexadecimal values.

2. Challenge Scenario

The sample is designed to behave differently when it detects a debugger, sandbox, or analysis environment.

During the analysis, we found checks for:

- Debugger presence
- Debug objects
- Hardware breakpoints
- Physical RAM
- Running process count
- Analysis tools
- USB storage history

The sample also prepares HTTP C2 communication and maintains a list of C2 hostnames. It builds a client fingerprint using information from the system.

The challenge gives us 15 questions, and the main task is to locate each value inside the binary.

3. Tools Used

- **Ghidra**
 - Ghidra Decompiler
 - Ghidra Listing
 - String Search
 - Cross References
 - Basic hexadecimal/ASCII/UTF-16 decoding
-

4. Q1 — DOS Header Magic

What we need to find

The decimal value compared against the DOS header magic of the embedded payload.

Analysis

In Ghidra, go to:

```
FUN_140014dd0
```

In the decompiler, look for the comparison with the DOS header:

```
if (*param_1 == 0x5a4d)
```

The value is:

```
0x5A4D
```

Convert it to decimal:

```
0x5A4D = 23117
```

Answer

23117

What is happening?

The malware is checking for the MZ signature of a PE file.

5. Q2 — WinHTTP URL Path

What we need to find

The URL path used by the WinHTTP requests.

Analysis

Go to:

`FUN_140002c70`

Find the `WinHttpOpenRequest` call:

```
lVar4 = (*DAT_140023ea8) (  
    lVar3,  
    DAT_140023d28,  
    DAT_140023d70,  
    0  
);
```

Here, `DAT_140023d70` is the URL path.

Follow `DAT_140023d70` to its initialization.

The string is decoded through:

```
FUN_140002250  
FUN_140002480
```

After decoding the value, we get:

```
/api/debut
```

Answer

```
/api/debut
```

What is happening?

The malware uses this path when making its HTTP C2 requests.

6. Q3 — Relocation Entry Type

What we need to find

The relocation type that the malware processes.

Analysis

Go to:

`FUN_140003240`

Look for:

```
uVar5 = *(ushort *)puVar21 >> 0xc;  
if ((uVar5 != 3) || ...)
```

The relocation type being checked is:

3

Answer

3

What is happening?

The function is processing relocation entries while handling a PE image.

7. Q4 — HTTP Verb

What we need to find

The HTTP method used for sending C2 data.

Analysis

Follow:

`DAT_140023d28`

to where it is initialized.

We find:

```
*puVar1 = 0x4f0050;puVar1[1] = 0x540053;
```

These values represent a UTF-16 string.

Decoding them gives:

```
POST
```

Answer

POST

What is happening?

The malware sends its C2 data using an HTTP POST request.

8. Q5 — NtGlobalFlag Check

What we need to find

The byte value checked against `PEB.NtGlobalFlag`.

Analysis

Go to:

```
FUN_140006080
```

Look for the PEB checks:

```
if ((*(char *) ((longlong)ProcessEnvironmentBlock + 2) == '\x01') ||  
    (*(char *) ((longlong)ProcessEnvironmentBlock + 0xbc) == 'p'))
```

The relevant check is:

```
PEB + 0xBC
```

and the value being compared is:

```
'p'
```

ASCII `p` is:

```
0x70
```

Answer

0x70

What is happening?

The malware is checking a debugger-related PEB value.

9. Q6 — NtQueryInformationProcess

What we need to find

The `ProcessInformationClass` used by the second relevant `NtQueryInformationProcess` call.

Analysis

Search for:

`NtQueryInformationProcess`

There are multiple calls.

Look at the second call involved in the debug-object check.

The `ProcessInformationClass` argument is:

`0x1E`

Answer

0x1E

What is happening?

The malware uses the native Windows API to check information related to debugging.

10. Q7 — GetThreadContext

What we need to find

The `ContextFlags` value used before `GetThreadContext`.

Analysis

Search for:

`GetThreadContext`

Look immediately before the call.

We find:

```
local_4b8 = 0x100010;
```

This is the value used for the context flags.

Answer

0x100010

What is happening?

The malware retrieves thread context information, which can be used to check hardware debugging registers.

11. Q8 — Decoy Process

What we need to find

The exact command line passed to `CreateProcess`.

Analysis

Search for:

`CreateProcess`

Follow the parameters passed to the API.

The command line is:

```
explorer.exe shell:RecycleBinFolder
```

Answer

```
explorer.exe shell:RecycleBinFolder
```

What is happening?

The malware starts Explorer with the Recycle Bin shell location as a decoy process.

12. Q9 — Physical RAM Check

What we need to find

The minimum physical RAM value used by the sandbox check.

Analysis

Search for:

```
GlobalMemoryStatusEx
```

The function fills a `MEMORYSTATUSEX` structure.

The malware compares the physical memory value against:

```
0x80000000
```

This is:

```
2 GiB
```

Answer

0x80000000

What is happening?

The malware checks the available physical memory as part of its environment/sandbox checks.

13. Q10 — USB Storage Registry Key

What we need to find

The registry path relative to HKLM.

Analysis

Go to:

`FUN_140004260`

The function builds a registry string from several hexadecimal DWORDs.

The values decode to:

`SYSTEM\ControlSet001\Enum\USBSTOR`

So the complete location is:

`HKLM\SYSTEM\ControlSet001\Enum\USBSTOR`

Answer

`SYSTEM\ControlSet001\Enum\USBSTOR`

What is happening?

The malware checks the Windows USB storage registry information.

14. Q11 — Process Count

What we need to find

The minimum number of running processes required before the extended process scan.

Analysis

Find the process enumeration code.

The relevant condition is:

```
if (uVar13 < 0x32)
```

Convert:

```
0x32 = 50
```

Answer

50

What is happening?

After reaching this process-count threshold, the malware performs its larger process-name scan.

15. Q12 — Analysis Tool Detection

What we need to find

The reverse-engineering/analysis tool process name included in the scan.

Analysis

Go to the section where the process-name list is created.

Look at:

```
local458[33]
```

The string is stored as encoded UTF-16 values.

Decode the values and the process name becomes:

```
cheat-engine-x86_64.exe
```

Answer

```
cheat-engine-x86_64.exe
```

What is happening?

The malware checks whether Cheat Engine is currently running.

16. Q13 — Command-Line Argument

What we need to find

The exact `argv[1]` value required by the early argument check.

Analysis

Find the command-line parsing code around:

```
GetCommandLineW()
```

When:

```
local_48 == 2
```

the malware creates the comparison string using:

```
0x7400730x7400610x630069
```

These values decode as UTF-16LE to:

```
static
```

Answer

```
static
```

What is happening?

The malware checks whether the supplied command-line argument is `static`.

17. Q14 — Plaintext C2 Hostname

What we need to find

The only C2 hostname stored in plaintext among the 20 entries.

Analysis

Go to:

```
FUN_140001f40
```

This function initializes the hostname table.

The table starts at:

```
DAT_140023ee0
```

The first entry is different from the others. Instead of being passed through the normal string-decryption function, it is copied directly.

The string is represented by:

```
0x6d6f7270  
0x7265766f  
0x6f2e6573  
0x6772
```

Decode the bytes as little-endian ASCII:

```
prom  
over  
se.o  
rg
```

This gives:

```
promoverse.org
```

Answer

```
promoverse.org
```

What is happening?

The malware has 20 C2 hostname entries, but the first one is stored directly instead of being processed by the encrypted-string routine.

18. Q15 — MachineGuid

What we need to find

The registry value name queried from:

```
HKLM\SOFTWARE\Microsoft\Cryptography
```

Analysis

Go to:

```
BuildClientFingerprintId
```

at:

```
0x140002A50
```

The hint tells us to look after the `CryptGenRandom` and IV-copy sections.

Continue until you find:

```
HeapAlloc
```

followed by:

```
strcpy
```

The string created by this `strcpy` is then passed to:

```
pRegQueryValueExA
```

The registry key being accessed is:

```
HKLM\SOFTWARE\Microsoft\Cryptography
```

The value name is:

```
MachineGuid
```

Answer

```
MachineGuid
```

What is happening?

The malware retrieves the machine's `MachineGuid` and uses it as part of the client fingerprint.

19. Final Answers

#	Answer
1	23117
2	/api/debut
3	3
4	POST
5	0x70
6	0x1E
7	0x100010
8	explorer.exe shell:RecycleBinFolder
9	0x80000000
10	SYSTEM\ControlSet001\Enum\USBSTOR
11	50
12	cheat-engine-x86_64.exe
13	static
14	promoverse.org
15	MachineGuid

20. What I Learned From This Challenge

The main thing I learned from Phantom Loader was how to approach a malware sample when the important information isn't sitting in plaintext.

The most useful Ghidra workflow was:

```
Find interesting API/string
↓
Open the function
↓
Check the arguments
↓
Follow the referenced variable
↓
Find where it is initialized
↓
Decode the value
↓
Confirm the answer
```

I also got more practice with:

- Reading decompiled C code
- Following Ghidra cross-references
- Understanding Windows API arguments
- Recognizing PEB-based anti-debugging
- Understanding process-based anti-analysis
- Decoding little-endian strings
- Decoding UTF-16LE strings
- Finding C2 configuration
- Identifying registry-based fingerprinting

Overall, the challenge showed how malware can hide fairly simple information behind dynamic string construction, encoded values and multiple checks. The important part was not understanding every line of the binary, but knowing **where to look and how to follow the data until it became readable**.